Title : Project Registration & Progress Review **FF No. 180**

Department: Computer Engineering	**Academic Year:** 2026-27
Semester : SEM- V	**Group No. : TY-A2**
Project Title: Explainable Feature Fusion and Ensemble Learning for Modi Script Recognition	
Project Area: Artificial Intelligence and Data Science	

Group Members Details:

Sr. No.	Class & Div.	Roll No.	G.R.No.	Name of Student	Contact No.	Email ID
1	CS-A	44	12413412	Ananya Viraj Choudhari	9307971404	viraj.ananya24@vit.edu
2	CS-A	61	12414120	Arya Bharat Patil	8605482342	arya.patil24@vit.edu

Name of Internal Guide: Prof. Nilam Honmane
Contact No. & Email ID: 98234 37739
nilam.honmane@vit.edu

Project approved / Not approved

Guide **Project Coordinator** **Head,**
Department of Computer Engineering

FF No 180

Project Synopsis

1. Introduction

Modi is the cursive script that was used to write Marathi for several centuries, mostly for land records, court papers and letters in Maharashtra. Very few people can read it today, so a lot of these documents just sit in archives. In this project we try to recognise single handwritten Modi characters using a computer. Instead of a deep network, we use three hand-made feature types (HOG, Zernike moments and LBP) with classical classifiers like SVM and Random Forest, and we use SHAP to see why the model picked a particular character. We also made a small web app where you can upload or draw a character and check the result.

2. Problem Statement

Most papers on Modi recognition use deep networks like CNN or VGG16 trained on about 90 images per character, and many of them show signs of overfitting, such as a big gap between training and validation accuracy. The few papers that used classical methods either gave raw pixels to the classifier or tested on very small private datasets. Almost none of them explain their predictions or say which characters get mixed up. So we wanted to check how far handcrafted features can go on a public dataset, and whether we can show why the model makes each decision.

3. Objectives

- To recognise 57 handwritten Modi characters using three handcrafted features, HOG, Zernike moments and LBP, instead of raw pixels. Kulkarni et al. (2014) showed that Zernike moments work for Modi digits, and Mandke et al. (2024) gave raw pixels to SVM and KNN, so we want to see what proper features can do on all the characters.
- To compare Random Forest, SVM and XGBoost on these features and see if combining them with stacking gives better accuracy than the best single one. Ekbote et al. (2022) used Random Forest and XGBoost for Modi, but on VGG16 features and one model at a time.
- To use SHAP to show which part of a character the model looked at, and to list the pairs of characters it confuses the most. Kolte and Gupta (2025) and Anam and Gupta (2015) both say look-alike characters are the main problem, but neither says which ones.
- To check how a simple CNN does compared to our model when both are trained on the same small amounts of data, starting from 72 images per character. Bhong and Kulkarni (2025) and Chandankhede et al. (2025) got high training accuracy with CNNs but much lower validation accuracy, which made us want to test this ourselves.

4. Methodology

We follow these steps, with a fixed random seed so the results can be repeated:

- Data Collection
  - We use the public MODI-HChar dataset from Kaggle, which has 57 character classes.
  - For each character, 200 images are kept aside only for testing, and the model is trained on different images.
- Data Pre-processing

  - Each image is converted to grayscale and then to black and white using Otsu thresholding.
  - We remove scan borders and small dots, then resize the character to 64 x 64 pixels.
- Segmentation of Multi-character Images
  - If an image has more than one character, we find the line and cut it where there are gaps between characters.
- Feature Extraction
  - HOG captures the direction of the strokes, and Zernike moments capture the overall shape.
  - LBP captures small patterns like stroke ends and joints.
- Feature Fusion and Dimensionality Reduction
  - The three feature sets are joined into one vector of 2,261 values.
  - LDA then reduces this to a much smaller number of features before classification.
- Classification
  - We train Random Forest, SVM and XGBoost, and then a stacking model that combines the three.
- Evaluation
  - We check accuracy, precision, recall and F1 score on the test images, and use the confusion matrix to find which characters get mixed up.
  - We also train a simple CNN on the same data to compare against.
- Explanation and Web App
  - SHAP shows which parts of the character pushed the model towards its answer.
  - A small web app lets you upload or draw a character and see the prediction and its explanation.

5. Hardware & Software

Hardware Requirements

- Processor: Intel Core i5 / AMD Ryzen 5 or above (CPU only; no GPU required)
- RAM: Minimum 8 GB (16 GB recommended for training)
- Storage: Minimum 20 GB of free space (dataset download and cached features)
- Internet Connection: Required only for the one-time dataset download

Software Requirements

- Operating System: Windows / Linux
- Programming Language: Python 3.11 or above
- Image Processing: OpenCV, scikit-image, mahotas
- Machine Learning: scikit-learn, XGBoost
- Explainability: SHAP
- Data Processing and Visualisation: NumPy, Pandas, Matplotlib
- Reference Baseline only: PyTorch (CPU)
- Application Development: Starlette (Python web framework) with HTML, CSS and JavaScript
- Development Environment: Jupyter Notebook / Visual Studio Code, Git

6. Environment

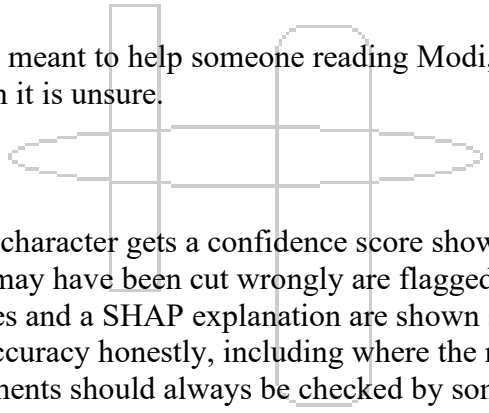
The project runs on a normal laptop CPU, so it doesn't need a GPU or any cloud server. Classical models train much faster than deep networks, and we save the processed images and features so they don't have to be computed again every time. Reading old documents digitally also means the fragile paper copies get handled less.

7. Sustainability

- We tried to keep the project easy to maintain and reuse.
- Project-specific considerations: The system is designed to be:
- Modular: each step (cleaning, features, models, explanations) is a separate file, so one part can be changed without touching the rest.
- Reproducible: a fixed random seed and fixed library versions give the same results on every run.
- Configurable: all the settings are in one config file.
- Lightweight: it runs on a normal laptop without a GPU.
- Tested: automatic tests check the main steps and the web app.

8. Safety

This is a student project meant to help someone reading Modi, not to replace them, so the app tries to make it obvious when it is unsure.



Safety Considerations

- Every predicted character gets a confidence score shown in green, amber or red.
- Characters that may have been cut wrongly are flagged.
- The top 3 guesses and a SHAP explanation are shown so the user can double-check.
- We report our accuracy honestly, including where the model fails.
- Important documents should always be checked by someone who can read Modi.

Cybersecurity

- Uploaded images are processed on the computer itself and are not saved.
- The app checks that an uploaded file is really an image and is not too large.
- Our Kaggle login details are kept out of GitHub.

9. Ethics

Modi documents are part of Maharashtra's history and can include land and family records, so we tried to be careful about how the results are presented.

Important Considerations

- Transparency: every prediction comes with an explanation of why the model chose it.

- Honest reporting: results are on a public test set, including the cases where our model did worse than expected.
- Data licence: the dataset is public under a CC0 licence.
- Bias: the dataset has a limited number of writers, so the model may not work as well on other handwriting styles.
- Human checking: the output is a helper, not a final reading.
- Labels: the Devanagari names of the characters still need to be checked by someone who reads Modi.

10. Cost

The project costs almost nothing. The dataset is free, all the software is open source, and everything runs on a laptop we already have.

Cost Factors

- Dataset: free (Kaggle, CC0 licence)
- Software: free and open source
- Hardware: our own laptop, no GPU
- Time spent on coding, experiments and writing
- Optional: hosting the web app online for a demo

Potential Economic Benefits
It could help reduce:

- The time it takes to read Modi documents by hand
- Dependence on the few people who can still read Modi
- The cost of digitising old archives
- The effort needed to learn the script

11. SDG Project Mapping

SDG 4 - Quality Education

- Can be used as a learning aid for reading Modi characters.
- Shows why the model chose a character, which helps learners see how similar characters differ.

SDG 9 - Industry, Innovation and Infrastructure

- Uses machine learning for digitising old documents.
- Runs on a normal CPU, so it is cheap to use.

SDG 11 - Sustainable Cities and Communities (Target 11.4: cultural heritage)

- Helps keep Maharashtra's old records readable.
- Can help archives digitise fragile documents.

12. Application and Future Scope

Application

- Archives: help libraries and archives read and sort Modi documents.
- Research: help historians read old land and court records.
- Education: a practice tool for students learning Modi.
- Transcription help: suggest characters with confidence scores for an expert to check.

Future Scope

- Handle joined-up cursive writing, where characters touch each other.
- Read whole words and lines and convert them to Devanagari.
- Add vowel signs (matras) and joint letters.
- Train on handwriting from more people and time periods.
- Try combining our features with features learned by a CNN.
-

FF No 180

Group No.	TY-A2		
Activity	Review Schedule	Progress Review Report submitted	Signature of Guide
Review 1	Mid Sem. Semester	Yes / No	
Review 2	End of Semester	Yes / No	

Format of Progress Review Report:

Review No.: 1	**Group No.: TY-A2**	**Date:**
Progress Review Report		

Signature of Guide:

Review No.: 2	**Group No.: TY-A2**	**Date:**

Progress Review Report

Signature of Guide: